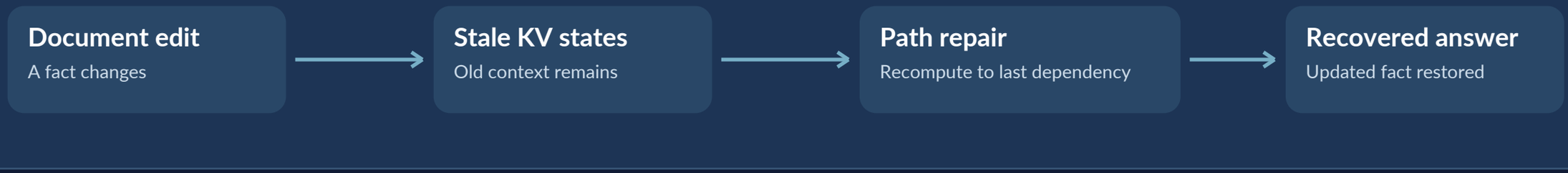
RESEARCH PRESENTATION

How Far Does an Edit Reach?

Repairing Derived Facts in Stale KV Caches

THE RESEARCH IDEA



PRESENTED BY

Nour Shreif • Hadi Ghazi • Karin Al Zoabi

Lebanese University

2026

THE PROBLEM + THE IDEA

How far does an edit reach?

Repairing derived facts in stale KV caches

Why a one-sentence edit can break later answers

DERIVED FACT: THE ANSWER IS NEVER IN THE EDITED SENTENCE

01 ORIGINAL DOCUMENT

Varnex uses DELTA

Delta maps to code Q7

Cached (stale) answer **Q7**

02 DOCUMENT EDIT

Varnex now uses LAMBDA

Lambda maps to code R6

Correct answer **R6**

03 OUR SOLUTION: THE PATH WINDOW

Recompute only the affected path



Edit — Last dependent sentence

Avoid re-reading the tail

Why the obvious fixes fall short

Recompute the edited sentence only — FAILS

Li: recovers almost none of the decision

Re-read the whole document — TOO COSTLY

Correct, but throws away the savings of caching

Fixed window after the edit — LIMITED BY DISTANCE

Mao: beats important tokens, but recovery collapses when dependent text is far

Our contributions

1. **Path window on derived facts**
   Recompute from the edit to the last dependent sentence; 3 models, 40 repair policies
2. **Finding the last dependent sentence**
   A free word index and language-model dependency graphs
3. **Where each approach breaks**
   Paraphrased text, and an attention-based competitor

CENTRAL QUESTION

How can we identify the **last sentence affected by an edit** and repair its cached dependencies **without recomputing the entire document?**

METHOD + EXPERIMENT

From edit to repaired cache

Many repair policies | three 7B–8B models | controlled WikiText-103 benchmark

1 **EDIT** Cache the document, then change one fact → 2 **FIND** Sentences affected by the edit → 3 **BOUNDARY** The last affected sentence → 4 **RECOMPUTE** Contiguous KV path: edit → boundary → 5 **EVALUATE** New answer recovered? Tokens recomputed?

FINDING THE BOUNDARY: TWO FINDERS

Both read only the text, never the cache, and return a stopping sentence.

FAST **1 Word index** no model, every edit

Words the edit changed: **Delta, Lambda** → Mark every later sentence containing either → Follow rare names & codes, up to **3 hops**

Both words matter: the Lambda row is newly relevant, but the Delta row is stale too.

SEMANTIC **2 GPT-4.1, for paraphrase** "the third plan" ↔ "Plan three"

ONCE, WHEN STORED
Builds a dependency graph → **old links**

AT EDIT TIME
Asked about the new version → **new links**

Stop at the furthest sentence either one reaches.

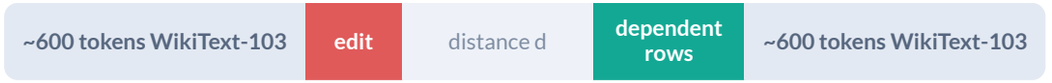
REPAIR POLICIES · 3 INSTRUCTION-TUNED MODELS

Qwen3-8B | Llama-3.1-8B | Mistral-7B

No repair (floor) · Full re-read (ceiling) · Mao: fixed windows · Mao: CarrierWindow · Li: appended correction · … and more

CONTROLLED BENCHMARK

9 settings × 50 documents = 450 / model



Direct · **derived** at d = 0 / 250 / 1,500 · **3-hop** · **fan-out** · **noisy** · **paraphrased** at d = 250 / 1,500

Rows in random order · every edit keeps the same token length in all three tokenizers

MARGIN RECOVERY **0** = no better than stale cache — **1** = as good as full re-read + answer accuracy + recomputed tokens

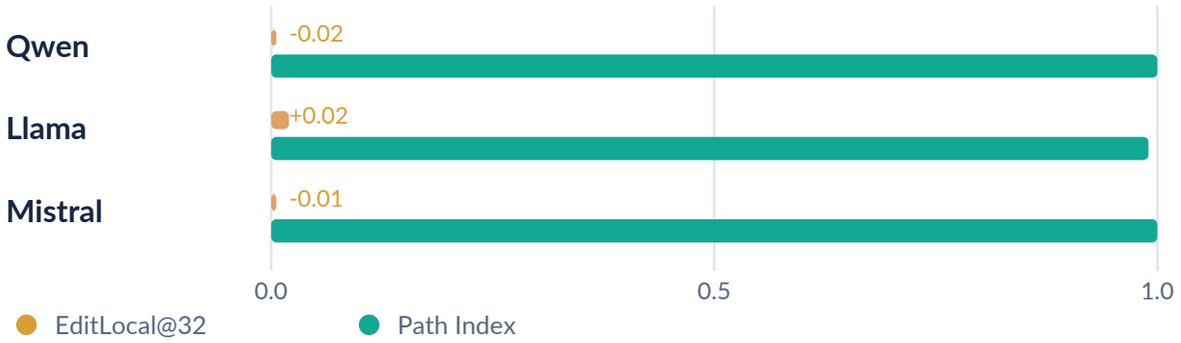
RESULTS + DEFENSE TAKEAWAY

What the experiments establish

Strong answer recovery when the correct repair endpoint is identified

DISTANT DERIVED DEPENDENCIES | d = 1,500

Margin recovery: EditLocal@32 vs. Path Index



EXPLICIT DEPENDENCIES

0.99-1.00

margin recovery (Path Index)

95-100%

answer accuracy on retained cases

Across 18 derived setting × model cells

RECOMPUTED TOKENS VS FULL RE-READ

d = 250

18% of full (368 / 2,008)

d = 1,500

50% of full (1,628 / 3,257)

PARAPHRASED DEPENDENCIES

Word index

0.11-0.42 MR

GPT-4.1, combined

0.84-0.98 MR

Attention@512: 0.98-1.03 MR (extra overhead excluded)